

九齿编译优化 T1-2-1 赛题报告

赛题编号：T1-2-1
规则版本：v0.6
适用赛事：2026 春季人工智能大赛
关联赛道：九齿开发赛道
小组名称：zhaxi
GitHub ID：zili2004
提交分支：2026-spring-zili2004-T1-2-1
报告日期：2026-07-12
选择的特化类别：Category 1（Contiguous Fast Path）+ Category 2（Divisible Tile Fast Path）
Baseline：InfiniTensor/ninetoothed@c9ebd4950a185beed8d4c1db9ff4a1fd133934ae
设备：RTX5090

1. 赛题理解与目标

NineToothed 基线以正确性和通用性为优先。在连续布局、整除分块等信息已经能够由 AOT variant 或运行时元数据可靠判定时，基线生成代码仍可能包含：

- 冗余边界 mask；
- 冗余多维 stride/offset 展开；
- 冗余 pointer arithmetic；
- 已知连续布局仍通过通用参数传递 size/stride；
- 可判定场景未命中专门的 AOT variant。

本提交选择以下两类受约束特化：

1. **Contiguous Fast Path**：当所有参与计算的 tensor 满足严格连续布局条件时，消除运行时 stride 参数和多维 stride 展开。
2. **Divisible Tile Fast Path**：当 tile 对有效迭代范围无尾块时，生成无边界 mask 的 load/store 路径。

设计原则是：特化条件必须可证明；条件不满足时保留原通用路径；不能通过 benchmark 名称、函数名或固定尺寸命中特化。

2. 改动范围

主要修改：

文件	主要内容
src/ninetoothed/generation.py	增加特化 hint；生成 contiguous/divisible 代码；删除失活 size/stride metadata；保留 launch grid 依赖参数
src/ninetoothed/aot.py	生成特化 variant；建立严格 dispatcher guard；增加 int32/int64 fallback；增加测试专用 actual-dispatch 记录
tests/test_aot_specialization.py	结构测试、实际命中测试、CUDA correctness、fallback correctness、overflow guard 测试
benchmarks/bench_aot_specialization.py	baseline/submitted 对比；median/P25/P75；actual dispatch；generated-source 统计；JSON 输出

未通过删除、跳过或弱化既有测试实现优化。测试和 benchmark 已拆分为两个独立文件。

3. 技术方案

3.1 AOT variant

提交代码生成以下关键路径：

- `flatten_contiguous_divisible`
 - 连续布局；
 - tile 无尾块；
 - 删除边界 mask；
 - 删除通用 stride 参数和 stride arithmetic。
- `flatten_contiguous_masked`
 - 连续布局；
 - 存在尾块；
 - 正确保留边界 mask；
 - 删除通用 stride 参数和大部分冗余地址计算。
- `legacy` / int64 fallback
 - 特化条件不满足时使用；
 - 支持非连续 tensor、广播 stride、复杂 view、标量参数和超出 int32 范围的元数据。

3.2 运行时启用条件

1D/2D fast path 仅在以下条件同时满足时启用：

- 参数为支持的 tensor-only elementwise 形态；
- 所有 tensor 的运行时 shape 兼容；
- 输入与输出均满足完整 contiguous 条件；
- size/stride 可安全表示为 int32；
- divisible variant 还需满足 tile 无尾块条件；
- masked variant 保留尾块 mask，不错误删除边界检查。

以下场景保守回到通用路径：

- 3D runtime；
- noncontiguous tensor；
- transpose、permute、as_strided；
- expand 产生的 stride-0 broadcast；
- scalar argument；
- size/stride 超出 int32 范围。

3.3 Unused metadata pruning

连续特化后，部分 shape/stride 参数不再被 Triton kernel 使用。提交实现基于 AST 的活跃引用分析，仅删除同时满足以下条件的 metadata：

- kernel body 不引用；
- launch grid 不引用。

该规则避免了错误删除 grid 计算仍需要的 size 参数。最终生成代码中，1D/2D fast path 的 stride 参数均从 kernel signature 中删除。

3.4 Actual dispatch 记录

通过测试专用 `NINETEETHED_DISPATCH_TRACE=1` 开关，生成的 dispatcher 记录实际执行的 variant。记录逻辑只提供可观察性，不改变 guard、variant 顺序或计算结果。正式生产路径默认不启用记录。

benchmark 不再根据场景名称中的 `divisible` 或 `masked` 推断 expected variant；expected variant 在结构化 case 定义中显式给出。

4. Weakness Analysis 摘要

完整分析见 `weakness.md`。核心弱势包括：

1. 连续且整除的 1D/2D 场景：基线仍生成 mask、stride 表达式和多余参数。
2. 连续但有尾块的 masked 场景：边界 mask 必须保留，但基线仍保留全部 stride 参数和通用多维 pointer 展开。

3. **AOT 已知布局未充分利用**：基线缺少可由 dispatcher 实际选择的 contiguous/divisible 专门路径。

5. 测试设计与正确性

5.1 测试类型

- generated-source 结构测试；
- 1D/2D divisible actual-dispatch 测试；
- 1D/2D masked actual-dispatch 测试；
- noncontiguous、scalar、broadcast、transpose、permute、as_strided fallback correctness；
- 3D runtime 保守回退；
- int64 overflow guard 结构测试。

5.2 最新 benchmark correctness

最新 JSON 共记录 **19 个 runtime 场景**：

- correctness: **19/19**；
- submitted actual-dispatch match: **19/19**；
- specialization hit: **10 个**；
- legacy/fallback: **9 个**；
- 所有场景 `max_diff = 0.0`。

实际命中的 submitted variant 包括：

- 1D contiguous divisible；
- 1D contiguous masked；
- 2D contiguous divisible；
- 2D contiguous masked。

3D、noncontiguous 和 scalar 场景均未误命中特化。

6. 参考资料与工具使用

6.1 关键参考

- NineToothed 框架源码：
 - `src/ninetoothed/generation.py`
 - `src/ninetoothed/aot.py`
 - `src/ninetoothed/tensor.py`
- NineToothed T1-2-1 赛题规则 v0.6。
- Triton Language Reference: <https://triton-lang.org/>
- PyTorch Tensor、stride、contiguous layout 相关文档。
- InfiniTensor / NineToothed 开源社区文档，以及仓库中的既有测试、构建脚本和代码生成流程。

本提交以赛事提供的 NineToothed 基线源码为基础。除上述开源项目、官方文档和 AI 辅助工具外，未复制其他参赛队伍的未授权实现。

6.2 AI 辅助声明

本提交在开发过程中使用了以下 AI 辅助工具：

- **Gemini pro**：用于代码库探索、baseline weakness 识别、特化条件讨论、测试框架搭建和部分文档初稿。
- **OpenAI ChatGPT**：用于实现分析、错误定位、测试与 benchmark 拆分、actual-dispatch 验证方案、结果整理和报告修订。

AI 辅助范围包括：

- 阅读和解释现有源码；
- 提供候选设计与修改建议；
- 辅助生成或重构部分代码；

所有 AI 生成或建议的内容均由参赛者人工审查、理解、修改并在本地环境中实际执行验证。

报告中的 benchmark 数据来自实际 GPU 运行生成的 JSON 文件，未人工编造或修改测量结果。

6.3 第三方工具

本提交使用的第三方工具和运行环境包括：

- Python 标准库：argparse、ast、json、pathlib、statistics、subprocess、time 等；
- PyTorch：构造 CUDA tensor、生成参考结果和检查正确性；
- Triton：生成和编译 GPU kernel；
- CUDA Runtime / NVIDIA Driver / NVCC：AOT 编译和 GPU 执行；
- pytest：运行新增测试与既有回归测试；
- Git：切换 baseline/submitted 源码版本并记录提交；
- Linux shell 工具：执行测试、构建和结果检查。

7. Benchmark 方法

测试设备：**NVIDIA GeForce RTX 5090**。

对比方式：baseline 使用指定基线源码，submitted 使用当前提交源码。

每个场景：

- warmup：50；
- 每组 repeat：500；
- 独立采样组数：9；
- 主指标：9 组结果的 median；
- 同时记录 P25、P75 和全部 samples。

使用 median 的原因是微秒级 kernel 容易受到 GPU 频率、系统调度和后台负载影响。JSON 中部分样本存在明显长尾，但 median 对异常值更稳健。

8. Runtime 结果

7.1 汇总

指标	结果
Overall median speedup	1.0000x
Specialization-hit median speedup	0.9982x
Fallback median speedup	1.0000x
Overall average speedup	0.9991x
Specialization-hit average speedup	0.9975x
Fallback average speedup	1.0009x

按维度：

维度	all median	hit median	fallback median
1D	0.9953x	0.9915x	0.9953x
2D	1.0110x	1.0072x	1.0186x
3D	1.0000x	不启用 runtime fast path	1.0000x

结论：总体 runtime 基本持平；2D hit median 为 **1.0072x**，表现较好；1D hit median 为 **0.9915x**，存在轻微回退。

7.2 全部 runtime 场景

场景	baseline ms	submitted ms	speedup	actual variant	hit
runtime_1d_none_flatten_divisible_bf16	0.008425	0.008550	0.9854x	flatten_contiguous_divisible	是
runtime_1d_none_flatten_masked_bf16	0.008467	0.008581	0.9867x	flatten_contiguous_masked	是
runtime_1d_default_flatten_divisible_bf16	0.008502	0.008533	0.9964x	flatten_contiguous_divisible	是
runtime_1d_default_flatten_masked_bf16	0.008436	0.008435	1.0001x	flatten_contiguous_masked	是
runtime_1d_wide_flatten_divisible_bf16	0.008490	0.008458	1.0038x	flatten_contiguous_divisible	是
runtime_1d_wide_flatten_masked_bf16	0.008378	0.008561	0.9786x	flatten_contiguous_masked	是
runtime_2d_none_flatten_divisible_bf16	0.009175	0.009145	1.0033x	flatten_contiguous_divisible	是
runtime_2d_none_flatten_masked_bf16	0.009187	0.009250	0.9932x	flatten_contiguous_masked	是
runtime_2d_square_flatten_divisible_bf16	0.009262	0.009161	1.0110x	flatten_contiguous_divisible	是
runtime_2d_square_flatten_masked_bf16	0.009294	0.009146	1.0162x	flatten_contiguous_masked	是
runtime_3d_none_flatten_divisible_bf16	0.009800	0.009660	1.0145x	legacy	否
runtime_3d_none_flatten_masked_bf16	0.009682	0.009678	1.0004x	legacy	否
runtime_3d_balanced_flatten_divisible_bf16	0.009749	0.009749	1.0000x	legacy	否
runtime_3d_balanced_flatten_masked_bf16	0.009705	0.009814	0.9889x	legacy	否
fallback_noncontiguous_1d_none_fp32	0.008498	0.008538	0.9953x	legacy	否
fallback_noncontiguous_1d_default_fp32	0.008542	0.008586	0.9949x	legacy	否
fallback_noncontiguous_2d_none_fp32	0.009301	0.009131	1.0186x	legacy	否
fallback_noncontiguous_3d_none_fp32	0.009733	0.009822	0.9909x	legacy	否
fallback_scalar_arg_fp32	0.007959	0.007923	1.0045x	legacy	否

最佳 hit 场景为 runtime_2d_square_flatten_masked_bf16，speedup 为 **1.0162x**；最明显的 hit 回退为 runtime_1d_wide_flatten_masked_bf16，speedup 为 **0.9786x**。报告如实披露该回退，不将 generated-code 改善等同于稳定 runtime 加速。

9. Generated Code Metric

8.1 代表性结果

场景	mask	stride	pointer	source bytes	kernel params
runtime_1d_none_flatten_divisible_bf16	6→0	9→0	19→10	3449→1577	9→4
runtime_1d_none_flatten_masked_bf16	6→6	9→0	19→16	3498→3108	9→6
runtime_2d_none_flatten_divisible_bf16	6→0	18→0	31→16	6421→2886	15→7
runtime_2d_none_flatten_masked_bf16	6→6	18→0	31→28	6516→5871	15→9

关键结论：

- 1D/2D divisible 路径：mask 减少 100%；
- 1D/2D fast path：stride 表达式减少 100%；
- divisible pointer arithmetic 减少约 47%~48%；
- divisible source bytes 减少约 54%~55%；
- 1D kernel 参数 9→4，2D kernel 参数 15→7；
- masked 路径正确保留边界 mask，但删除所有 stride 参数。

`source_line_count` 在这些简单 kernel 中基本不变，因此本提交的主要 generated-code 收益体现在 mask、stride、pointer、source bytes 和参数数量，而不是行数。

8.2 3D source-only diagnostics

3D 仍生成可检查的 source variant，但 runtime dispatcher 保守回退：

- 3D divisible source：mask 6→0、stride 27→0、pointer 43→22、bytes 10440→5236；
- 3D masked source：mask 6→6、stride 27→0、pointer 43→40、bytes 10431→9678。

这些数据只用于 generated-source 结构证明，**不计为 3D runtime specialization hit**。

10. Fallback 设计与证明

场景	预期	实际	正确性
3D contiguous	legacy	legacy	通过
1D/2D/3D noncontiguous	legacy	legacy	通过
scalar argument	legacy	legacy	通过
stride-0 broadcast	legacy/fallback	legacy/fallback	测试覆盖
transpose/permute/as_strided	legacy/fallback	legacy/fallback	测试覆盖
int32 overflow	int64 fallback	dispatcher guard 存在	结构测试覆盖

fallback 的 mask、stride、pointer 和参数数量基本与 baseline 保持一致，说明提交没有通过破坏通用路径来换取局部指标。

11. 性能回退与未覆盖场景

10.1 已知 runtime 回退

- 1D hit median：0.9915x；
- `runtime_1d_wide_flatten_masked_bf16`：0.9786x；
- 整体 hit median：0.9982x。

这些 kernel 的单次时间约为 0.008~0.009 ms，dispatcher、launch 和系统波动占比较高。当前提交的主要收益是生成代码结构和有效 coverage，而不是所有微型 kernel 的稳定 runtime 加速。

10.2 未覆盖场景

- 3D runtime fast path；
- scalar/broadcast fast path（未选择 Category 3）；
- 任意复杂非连续布局；
- 超出 int32 范围的 fast path。

这些场景保留 legacy 或 int64 fallback，以优先保证隐藏 correctness 门槛。

12. 合规性与可复现性

- 生产代码不读取 benchmark 名称、测试名称或函数名称来决定特化；
- 生产代码不匹配报告中的固定输入尺寸；
- benchmark 的 fixed shape 仅作为测量输入；
- expected variant 由 case 元数据显式给出，不通过字符串名称推断；
- 未删除、跳过或弱化既有测试；
- 无联网运行依赖、闭源运行依赖或私有服务依赖；
- actual-dispatch trace 为测试专用 instrumentation，不改变计算语义。

13. 复现命令

```
python -m py_compile \  
  src/ninetoothed/aot.py \  
  src/ninetoothed/generation.py \  
  tests/test_aot_specialization.py \  
  benchmarks/bench_aot_specialization.py
```

```
NINETOOTHED_DISPATCH_TRACE=1 \  
PYTHONPATH=src \  
pytest -q tests/test_aot_specialization.py -s -ra
```

```
NT_BENCH_SAMPLES=9 \  
PYTHONPATH=src \  
python benchmarks/bench_aot_specialization.py --benchmark
```

结果输出：

```
benchmarks/bench_aot_speedup_compare_results.json
```

14. 总结

本提交完成了从 baseline weakness 识别、受约束特化、dispatcher guard、fallback 保证、actual-dispatch 证明、generated-source 测量到 median benchmark 的完整闭环。

当前结果表明：

- 19/19 runtime correctness 通过；
- 19/19 submitted dispatch 与预期一致；
- 10 个 1D/2D contiguous/divisible 或 masked 场景实际命中特化；
- divisible 路径完全删除 mask 和 stride 表达式；
- pointer arithmetic 与源码大小显著下降；
- fallback 不误命中且总体性能稳定；
- runtime 总体持平，2D 略有收益，1D 存在轻微回退并已如实披露。